

# Week 1 · Laboratory Problems — build the open loop in Simulink

- Course: Sensor Signal Processing and Fusion · Department of Autonomous Vehicle System Engineering
- Time: **one hour**, immediately after the Week 1 lecture hour
- Three problems, in order. Each one starts where the previous one stopped.

## What this hour is for, and why the models are not simply given

The lecture builds `w01_openloop.slx` **from a script**. Running one command produces a finished model, and reading that script is a good way to learn what the finished model contains. It is not a good way to learn how to build one.

This hour reverses that. The hull is provided, because integrating `otter.m` is not the subject of Week 1 and cannot be assembled from library blocks in an hour. Everything else — what the command is, how it reaches the plant, which of the twelve states matter, how a result is logged — is built by hand.

Every problem ends with a **number that was measured in the lecture**. A model that reproduces that number is doing the same physics as the lecture's model, whatever it looks like on the canvas. There is no single correct diagram.

## Before starting

```
cd lectures/w01_simulink/problems
w01_P1_start           % creates w01_P1.slx – the hull, and nothing else
```

The solver is already set to fixed-step `ode4` at `h = 0.02` s. **Do not change it**. The numbers below were measured with those settings.

Check work at any time:

```
w01_check(1)           % or w01_check(1, 'w01_P1_yourname')
```

The checker runs the model, compares it with the lecture, and prints `PASS` or `FAIL` for each test together with the value it expected.

### ★ One requirement on every model

The model must contain **one To Workspace block named `xlog`**, with Save format `Structure With Time`, fed by the plant's twelve-state output.

That is the only thing the checker needs. Requiring the raw twelve states rather than a tidy six-column log is deliberate: picking  $u$ ,  $v$ ,  $r$ ,  $x$ ,  $y$  and  $\psi$  out of the vector is the part of Week 1 worth being able to do without looking it up.

## Problem 1 · The open loop (25 minutes)

**Build.** A constant command into the hull, and a log coming out.

```
Constant [n0 ; n0] → Otter USV → To Workspace (xlog)
```

**Predict before running.** At steady state the thrust balances linear surge damping:

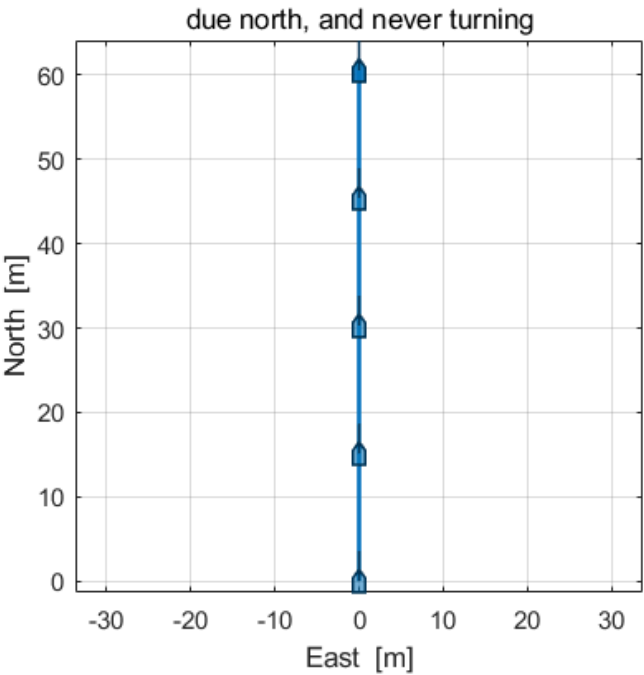
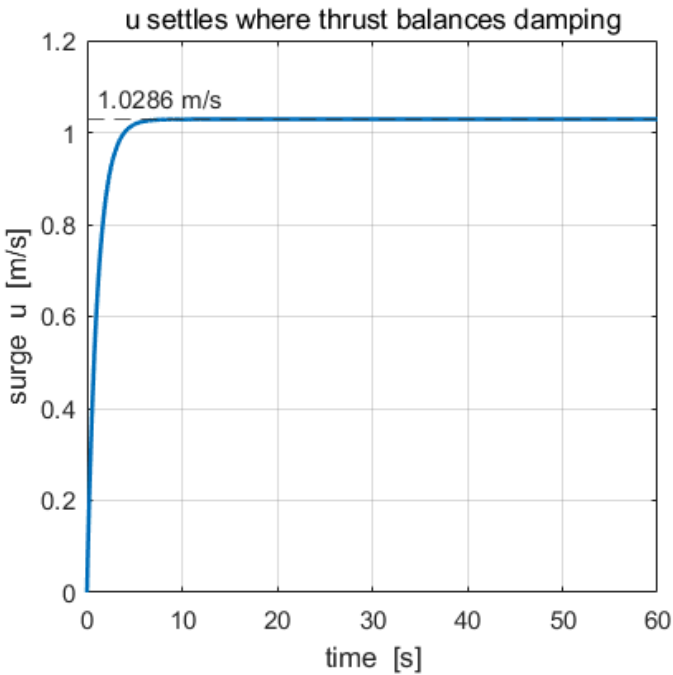
$$2\,k_{pos}\,n\,|n| = X_u\,u \quad \implies \quad u = \frac{2(0.01108)(60)(60)}{77.554}$$

Write the number down before pressing Run.

**Verify.** `w01_check(1)` — terminal  $u$ , and that  $v$  and  $r$  stay at zero.

| What the checker expects |            |
|--------------------------|------------|
| terminal surge speed     | 1.0286 m/s |
| sway velocity $v$        | exactly 0  |
| yaw rate $r$             | exactly 0  |

**What a correct model produces**



| Reading the figure |                                                                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| left               | $u$ rises with one time constant and settles on the dashed line at 1.0286 m/s. It does not overshoot: there is no loop, so nothing can |
| right              | the track is a straight line due north, and every hull silhouette on it points the same way                                            |
| the check          | if the track bends at all, the two Constant entries are not equal                                                                      |

**The point.** Two propellers, equal shaft speeds. The yaw moment is  $N = y_p (T_{left} - T_{right})$ , so equal speeds produce no turn at all, and the run is a test of surge alone. The command is a **2-vector**: `otter.m` expects  $n = [n_L ; n_R]$ , and a scalar is accepted by the Constant block and rejected by the plant.

## Problem 2 · The manoeuvre (20 minutes)

**Build.** Replace the constant by a schedule of time: straight, port, straight, starboard, straight.

```
Clock → MATLAB Function → Reshape (1-D array) → Otter USV → xlog
```

**Decide the signs first.** This is the whole problem, and it is decided before any block is placed:

$$N = y_p (T_{left} - T_{right}), \quad y_p = 0.395 \text{ m}$$

More thrust on the **left** turns the bow to **starboard**. A port turn therefore **slows the left propeller**:

| phase        | command                                 |
|--------------|-----------------------------------------|
| straight     | $n = [n_0 ; n_0]$                       |
| to port      | $n = [n_0 - \delta n ; n_0 + \delta n]$ |
| to starboard | $n = [n_0 + \delta n ; n_0 - \delta n]$ |

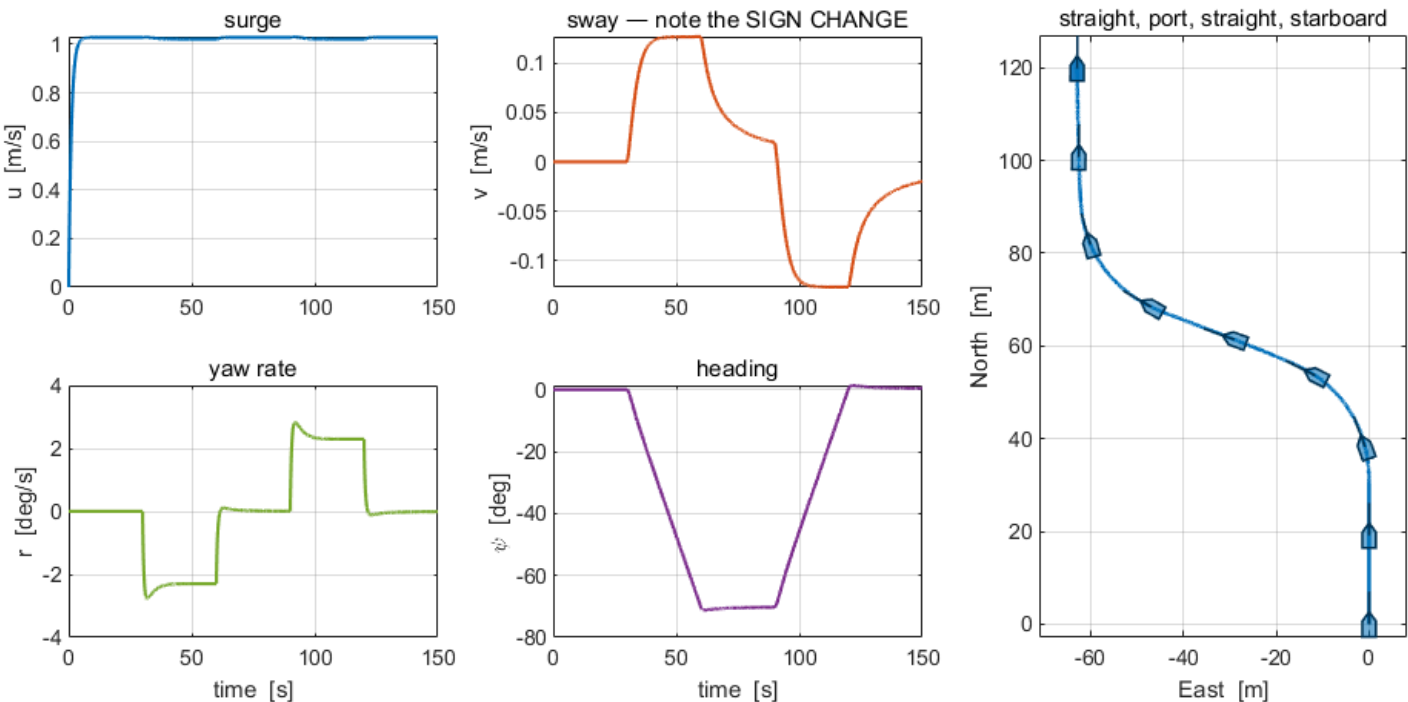
Nothing ever goes astern. At  $n_0 = 60$  and  $\delta n = 3.5$  the slower propeller still pushes ahead at **35.37** N while the faster one pushes at **44.68** N.

**Verify.** `w01_check(2)` .

| What the checker expects            |                                        |
|-------------------------------------|----------------------------------------|
| yaw rate in the port turn           | <b>-2.2942</b> deg/s                   |
| heading change, port then starboard | <b>-70.2</b> then <b>+70.6</b> deg     |
| sway $v$ in the two turns           | <b>+0.1264</b> then <b>-0.1264</b> m/s |

**Getting the sign backwards is visible immediately:** the heading change comes out **+70** deg where the checker wants **-70** deg.

**What a correct model produces**



| Reading the figure |                                                                                                                                       |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| $u$                | barely moves. The turns cost about <b>0.007</b> m/s of speed and nothing else                                                         |
| $v$                | rises to <b>+0.126</b> in the first turn and falls to <b>−0.126</b> in the second. <b>The sign change is the point of the problem</b> |
| $r$                | two flat plateaus of opposite sign, one per turn, with the transient at each phase boundary                                           |
| $\psi$             | a ramp down to <b>−70</b> deg, a hold, then a ramp back. A ramp UP first means the sign is reversed                                   |
| track              | the S-shape, with the hull drawn along it so the heading is visible where the track alone would not show it                           |

### ⚠ The Reshape block is not decoration

A MATLAB Function block whose output is written `n = [nL; nR]` produces a **2 × 1 matrix** signal. The plant does not care. The log does: one matrix input makes the whole To Workspace record three-dimensional, and the checker then finds one column where it expects twelve.

**The point.**  $\mathbf{Y}$  is exactly zero at every instant — both propellers face forward, so no combination of them has a component across the hull. Yet  $v$  is non-zero in **both** turns and **changes sign** between them. That sway is produced by the hull rotating, through the Coriolis term, and not by any side force. It is the crab angle, and Week 3 has to steer around it.

## Problem 3 · The vessel in a current (15 minutes)

**Build.** Nothing. The model of Problem 1 is used unchanged.

That is the exercise: work out **why** nothing needs to be added before running anything.

**Read** `otter.m` **lines 79–81 and line 204.**

$$u_c = V_c \cos(\beta_c - \psi), \quad v_c = V_c \sin(\beta_c - \psi), \quad \nu_r = \nu - [u_c \ v_c \ 0 \ 0 \ 0]^T$$

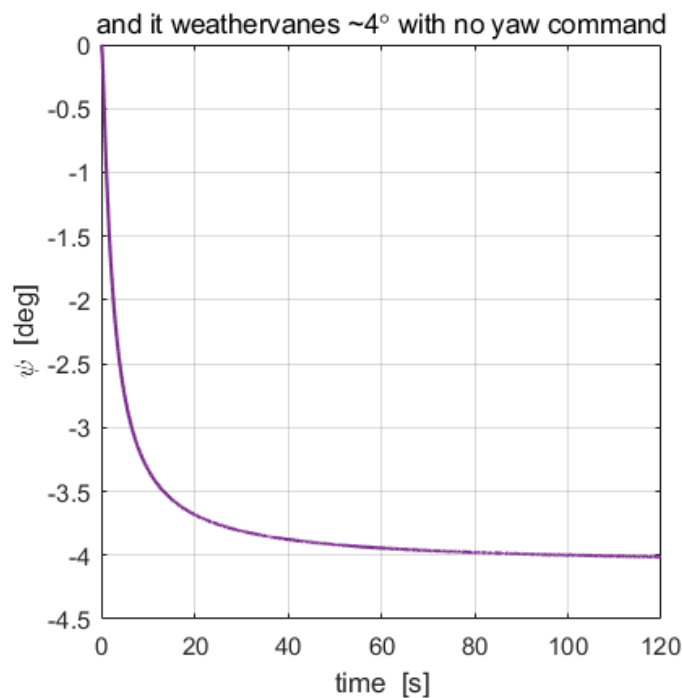
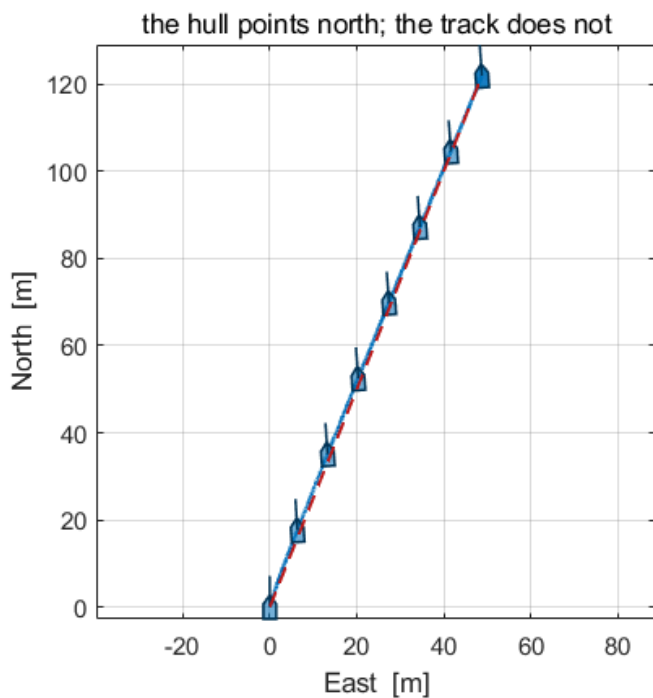
$$\dot{\eta} = \mathbf{J}(\eta) \nu \quad \text{— with } \nu, \text{ not } \nu_r$$

**Predict before running.** With  $V_c = 0.5$  m/s on the beam and the vessel making **1.0286** m/s through the water, the velocity over the ground is the vector sum, so the drift is roughly  $\arctan(0.5/1.0286) = 25.9^\circ$ .

**Verify.** `w01_check(3)` — it sets  $V_c = 0.5$  m/s and  $\beta_c = 90^\circ$  itself.

| What the checker expects   |                   |
|----------------------------|-------------------|
| ground speed               | <b>1.1091</b> m/s |
| track direction from North | <b>21.804</b> deg |
| drift, track minus heading | <b>25.809</b> deg |

**What a correct model produces**



| Reading the figure |                                                                                                                                                                               |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| left               | the hull silhouettes point <b>north</b> while the track leans <b>east</b> . The dashed red line is the straight track from start to finish, at <b><math>21.8^\circ</math></b> |
| right              | $\psi$ drifts about $-4^\circ$ over the run, with no yaw command anywhere in the model                                                                                        |
| the check          | if the hull silhouettes lean over with the track, $\psi$ is being drawn from the wrong state — the heading is index 12, not the direction of travel                           |

**The point.** A current is a **velocity, not a force**. Every hydrodynamic term is computed from  $\mathbf{v}_r$ , the velocity through the water, so the forces on the hull do not change. The position integrates  $\mathbf{v}$ , so the vessel is carried along by the water. That single asymmetry is why a current moves a vessel without pushing it.

Two things are worth noticing beyond the numbers:

- The predicted drift of  $25.9^\circ$  and the measured  $25.81^\circ$  agree, but the **ground speed is lower** than the naive vector sum of **1.144** m/s. The hull weathervanes about  $4^\circ$  into the flow, which turns part of the current from beam-on to head-on.
- **The vessel turns with no yaw command at all.** Cross-flow drag acts on  $\mathbf{v}_r$ , and its line of action does not pass through the origin, so it makes a yaw moment. Nothing in the model was asked to do this.

## Marking

|           | Weight | What is being marked                                                                                         |
|-----------|--------|--------------------------------------------------------------------------------------------------------------|
| Problem 1 | 30     | <code>w01_check(1)</code> passes, and the predicted <b>1.0286</b> m/s was written down <b>before</b> running |
| Problem 2 | 40     | <code>w01_check(2)</code> passes; the sign argument for the port turn is stated in one sentence              |
| Problem 3 | 30     | <code>w01_check(3)</code> passes; the answer to "why was nothing added to the model" is stated               |

A model that fails a check but whose written reasoning is right earns more than one that passes with no reasoning. The numbers are there to make the reasoning testable, not to replace it.

## If something goes wrong

| Symptom                                                           | Cause                                                                      | Fix                                                                      |
|-------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------|
| The model has no To Workspace block whose variable name is xlog   | the block exists but its <b>variable name</b> is still <code>simout</code> | open it and set the variable name to <code>xlog</code>                   |
| xlog is not a Structure With Time                                 | Save format left at <code>Array</code> or <code>Timeseries</code>          | set Save format to <code>Structure With Time</code>                      |
| xlog has 1 column. Feed the To Workspace the FULL 12-state vector | a matrix signal reached the log                                            | insert a Reshape set to <code>1-D array</code> after the MATLAB Function |
| Dimension error at the plant input                                | the command is a scalar                                                    | the Constant must be <code>[n0 ; n0]</code>                              |
| Heading changes the wrong way in Problem 2                        | the port turn speeds the left propeller up                                 | port turn <b>slows</b> the left one —<br>$N = y_p(T_{left} - T_{right})$ |
| Everything runs but every number is slightly off                  | the solver was changed                                                     | fixed-step <code>ode4</code> , <code>h = 0.02</code> s                   |

Reference solutions are in `../solutions/` . Read them **after** attempting the problem: each one explains not only what to build but why the alternatives were rejected.